

Modelado del sistema de comandas para restaurante

Proyecto: Comandas Web

Tecnología objetivo: Django

Contexto: taller básico intensivo de 4 horas

Versión del documento: 1.1

Este documento define una versión mínima viable (MVP) de una aplicación web para administrar las comandas de un restaurante. El alcance busca que el proyecto pueda desarrollarse durante una clase y, al mismo tiempo, permita practicar los conceptos centrales de Django: proyecto y aplicaciones, modelos, migraciones, administración, formularios, vistas, plantillas, URL y relaciones entre entidades.

1. Introducción

1.1 Planteamiento del problema

En un restaurante pequeño, las comandas suelen registrarse en papel o comunicarse verbalmente. Esto dificulta saber qué mesero atiende cada mesa, qué productos se han solicitado, cuáles mesas están disponibles y cuánto debe cobrarse al finalizar el servicio.

1.2 Solución propuesta

Se desarrollará una aplicación web que permita:

- Registrar meseros, mesas, categorías y productos.
- Seleccionar un mesero activo, sin implementar inicio de sesión.
- Consultar el estado de las mesas.
- Abrir una comanda para una mesa disponible.
- Agregar productos y cantidades a una comanda abierta.
- Calcular automáticamente subtotales y total.
- Cerrar la comanda y liberar la mesa.
- Mostrar un ticket con el desglose del consumo.

En este documento, **pedido** y **comanda** describen la misma operación. Se empleará el término **Comanda** en el modelo para mantener un vocabulario uniforme.

1.3 Objetivo general

Construir un sistema web sencillo que controle el ciclo de atención de una mesa, desde la asignación del mesero y la captura de productos hasta el cierre de la cuenta y la generación del ticket.

1.4 Objetivos didácticos

Al finalizar el taller, el participante podrá:

- Identificar la estructura de un proyecto Django.
- Crear modelos relacionados y ejecutar migraciones.
- Administrar catálogos desde Django Admin.

- Implementar vistas, rutas, formularios y plantillas.
- Aplicar reglas de negocio simples.
- Consultar relaciones y calcular valores derivados.

1.5 Alcance del MVP

El MVP incluye catálogos básicos, selección de mesero, apertura de una comanda por mesa, captura de productos, consulta de la cuenta, cierre de la comanda y visualización de un ticket.

No incluye autenticación para la interfaz operativa, roles, inventario, cocina, reservaciones, división de cuenta, impuestos configurables, propinas, pagos electrónicos ni impresión física. Django Admin sí conserva su autenticación incorporada para proteger la administración de catálogos. Los demás elementos pueden proponerse como extensiones posteriores.

1.6 Actores

- **Mesero:** selecciona su identidad, abre y actualiza comandas, consulta la cuenta y cierra el servicio.
- **Administrador:** mantiene los catálogos de meseros, mesas, categorías y productos mediante Django Admin.

1.7 Supuestos y reglas generales

- Solo puede existir una comanda abierta por mesa.
- Una mesa se considera ocupada cuando tiene una comanda abierta.
- El mesero activo se conserva en la sesión del navegador; esto no representa autenticación.
- Solo pueden agregarse productos activos.
- Solo pueden abrirse comandas en mesas activas.
- Un producto está disponible para captura únicamente si tanto el producto como su categoría están activos.
- Una mesa con una comanda abierta no puede desactivarse hasta cerrar el servicio.
- La selección del mesero identifica al responsable de una nueva comanda, pero no funciona como autorización: en el MVP cualquier mesero seleccionado puede consultar o modificar una comanda abierta.
- El precio usado en una comanda se copia al detalle al momento de agregar el producto. Así, un cambio posterior de precio no altera tickets históricos.
- Si se vuelve a agregar un producto que ya está en la comanda, se incrementa la cantidad y se conserva el precio unitario del detalle existente.
- Una comanda cerrada ya no puede modificarse.
- El total se obtiene sumando los subtotales de sus detalles.
- Cerrar una comanda registra la fecha de cierre y vuelve a dejar disponible la mesa.
- En el MVP, la inmutabilidad histórica cubre cantidades, precios y fechas. Los nombres de productos, meseros y mesas se consultan desde sus catálogos y podrían reflejar cambios posteriores; almacenar copias de esos nombres queda como extensión.

2. Procesos de Negocio

2.1 Administración de catálogos

Responsable: administrador.

Entrada: datos de meseros, mesas, categorías y productos.

Flujo:

1. El administrador ingresa a Django Admin.
2. Registra o modifica los catálogos.
3. Activa o desactiva registros según sea necesario.
4. El sistema valida y almacena la información.

Resultado: los catálogos quedan disponibles para la operación del restaurante.

2.2 Selección del mesero activo

Responsable: mesero.

Entrada: mesero seleccionado.

Flujo:

1. El sistema muestra los meseros activos.
2. El usuario selecciona su nombre.
3. El sistema guarda el identificador del mesero en la sesión.
4. El sistema muestra el tablero de mesas.

Resultado: las acciones siguientes quedan asociadas al mesero activo.

2.3 Apertura de una comanda

Responsable: mesero.

Precondiciones: existe un mesero activo y la mesa está disponible.

Flujo:

1. El mesero selecciona una mesa disponible.
2. El sistema comprueba que la mesa no tenga otra comanda abierta.
3. El sistema crea una comanda con estado **ABIERTA**, fecha de apertura, mesa y mesero.
4. La mesa se presenta como ocupada.

Resultado: la mesa tiene una comanda abierta lista para recibir productos.

2.4 Captura y modificación del consumo

Responsable: mesero.

Precondición: la comanda está abierta.

Flujo principal:

1. El sistema muestra los productos activos, opcionalmente agrupados por categoría.
2. El mesero selecciona un producto y una cantidad.
3. El sistema copia el precio vigente al detalle.
4. Si el producto ya existe en la comanda, incrementa su cantidad; de lo contrario crea un detalle.
5. El sistema recalcula y muestra el total.

Flujos alternos: el mesero puede cambiar la cantidad o eliminar un detalle antes del cierre. Una cantidad menor que uno o un producto inactivo se rechazan.

Resultado: el consumo de la mesa queda actualizado.

2.5 Consulta de cuenta

Responsable: mesero.

Flujo:

1. El mesero abre la comanda de la mesa.
2. El sistema muestra mesa, mesero, fecha, productos, cantidades, precios, subtotales y total.
3. El mesero verifica la información con el cliente.

Resultado: se presenta una vista previa del ticket sin cerrar la comanda.

2.6 Cierre de servicio y generación del ticket

Responsable: mesero.

Precondiciones: la comanda está abierta y contiene al menos un detalle.

Flujo:

1. El mesero solicita cerrar la cuenta.
2. El sistema pide confirmación.
3. El sistema calcula el total definitivo a partir de los detalles; no lo almacena en el MVP.
4. Cambia el estado a **CERRADA** y registra la fecha de cierre.
5. La mesa vuelve a mostrarse disponible.
6. El sistema presenta el ticket.

Resultado: se conserva un registro histórico inmutable del servicio.

2.7 Estados principales

Elemento	Estado	Significado
Comanda	ABIERTA	Acepta altas, cambios y eliminación de detalles.
Comanda	CERRADA	El servicio terminó y el registro es de consulta.
Mesa	Disponible	No posee una comanda abierta.
Mesa	Ocupada	Posee una comanda abierta.

El estado de la mesa es **derivado** y no necesita almacenarse: se calcula comprobando si existe una comanda abierta asociada. Esto evita inconsistencias entre una bandera de ocupación y las comandas reales.

3. Casos de Uso

3.1 Resumen

ID	Caso de uso	Actor principal	Resultado esperado
CU-01	Gestionar catálogos	Administrador	Catálogos disponibles y actualizados.
CU-02	Seleccionar mesero activo	Mesero	Mesero almacenado en la sesión.

ID	Caso de uso	Actor principal	Resultado esperado
CU-03	Consultar mesas	Mesero	Tablero con mesas disponibles y ocupadas.
CU-04	Abrir comanda	Mesero	Comanda abierta vinculada a mesa y mesero.
CU-05	Agregar producto	Mesero	Producto incorporado al consumo.
CU-06	Modificar detalle	Mesero	Cantidad actualizada y total recalculado.
CU-07	Eliminar detalle	Mesero	Producto retirado de la comanda.
CU-08	Consultar cuenta	Mesero	Desglose y total visibles.
CU-09	Cerrar comanda	Mesero	Servicio finalizado y mesa liberada.
CU-10	Ver ticket	Mesero	Ticket histórico disponible.

3.2 Especificación de casos principales

CU-02. Seleccionar mesero activo

- **Precondición:** existe al menos un mesero activo.
- **Disparador:** el usuario abre la pantalla de selección.
- **Flujo normal:** selecciona un mesero; el sistema valida que esté activo, guarda su ID en sesión y redirige al tablero.
- **Excepción:** si el registro no existe o está inactivo, se muestra un error y no se modifica la sesión.
- **Postcondición:** existe un mesero activo en la sesión.

CU-04. Abrir comanda

- **Precondiciones:** existe un mesero activo y la mesa está disponible.
- **Disparador:** el mesero selecciona la opción de atender una mesa.
- **Flujo normal:** el sistema valida la disponibilidad, crea la comanda y abre su pantalla de captura.
- **Excepción:** si la mesa ya está ocupada, se abre la comanda existente en modo consulta/edición o se informa el conflicto.
- **Postcondición:** hay una comanda **ABIERTA** para la mesa.

CU-05. Agregar producto

- **Precondiciones:** la comanda está abierta y el producto está activo.
- **Flujo normal:** el mesero elige producto y cantidad; el sistema guarda precio, cantidad y subtotal; después presenta el total actualizado.
- **Excepciones:** una cantidad inválida o un producto inactivo producen un mensaje de validación.
- **Postcondición:** el detalle queda agregado o acumulado.

CU-09. Cerrar comanda

- **Precondiciones:** la comanda está abierta y contiene productos.
- **Flujo normal:** el mesero revisa la cuenta, confirma el cierre y el sistema registra el estado y la fecha de cierre, calcula el total a partir de los detalles y después muestra el ticket.

- **Excepción:** si otro proceso ya cerró la comanda, no se realizan cambios y se muestra el ticket existente.
- **Postcondiciones:** la comanda queda cerrada, no admite cambios y la mesa queda disponible.

3.3 Diagrama de casos de uso

La fuente única y editable está en [diagramas/casos_de_uso.puml](#). Debe renderizarse desde ese archivo para evitar mantener copias divergentes.

4. Requisitos Funcionales

4.1 Catálogos

- **RF-01.** El sistema permitirá registrar, consultar, editar, activar y desactivar meseros.
- **RF-02.** El sistema permitirá registrar, consultar, editar, activar y desactivar mesas.
- **RF-03.** El sistema permitirá registrar, consultar, editar, activar y desactivar categorías.
- **RF-04.** El sistema permitirá registrar, consultar, editar, activar y desactivar productos.
- **RF-05.** Cada producto deberá pertenecer a una categoría y tener nombre y precio no negativo.

4.2 Operación

- **RF-06.** El sistema mostrará únicamente meseros activos en la pantalla de selección.
- **RF-07.** El sistema conservará el mesero activo en la sesión del navegador.
- **RF-08.** El usuario podrá cambiar de mesero activo.
- **RF-09.** El sistema mostrará un tablero con todas las mesas activas y su estado calculado. Una mesa inactiva que conserve una comanda abierta deberá seguir visible hasta cerrar el servicio como medida defensiva.
- **RF-10.** El sistema permitirá abrir una comanda únicamente para una mesa disponible.
- **RF-11.** La comanda registrará mesa, mesero, estado y fecha de apertura.
- **RF-12.** El sistema impedirá que una mesa tenga más de una comanda abierta.
- **RF-13.** El sistema permitirá agregar a una comanda abierta productos activos cuya categoría también esté activa, usando cantidades enteras positivas.
- **RF-14.** El detalle conservará el precio unitario vigente al momento de su creación.
- **RF-15.** Si se vuelve a agregar el mismo producto, el sistema podrá acumular la cantidad en el detalle existente.
- **RF-16.** El sistema permitirá modificar la cantidad o eliminar un detalle mientras la comanda esté abierta.
- **RF-17.** El sistema calculará el subtotal como $\text{cantidad} \times \text{precio_unitario}$.
- **RF-18.** El sistema calculará el total como la suma de los subtotales.
- **RF-19.** El sistema permitirá consultar la cuenta antes de cerrarla.
- **RF-20.** El sistema permitirá cerrar una comanda abierta que contenga al menos un detalle.
- **RF-21.** Al cerrar la comanda, el sistema registrará la fecha de cierre e impedirá modificaciones posteriores.
- **RF-22.** Al cerrar la comanda, la mesa quedará disponible automáticamente.
- **RF-23.** El sistema generará una vista de ticket con folio, mesa, mesero, fechas, detalles y total.
- **RF-24.** El sistema permitirá volver a consultar el ticket de una comanda cerrada.

4.3 Reglas de validación

- Los nombres de meseros, categorías y productos no pueden estar vacíos.

- El número o nombre visible de cada mesa debe ser único.
- El precio de un producto debe ser mayor o igual a cero.
- La cantidad de un detalle debe ser un entero mayor que cero.
- No se pueden agregar, modificar ni eliminar detalles de una comanda cerrada.
- Una mesa inactiva no admite nuevas comandas y una mesa ocupada no puede desactivarse.
- Una comanda **ABIERTA** debe tener **fecha_cierre** nula; una **CERRADA** debe tenerla informada y no anterior a **fecha_apertura**.
- El cierre debe realizarse como una operación atómica para evitar estados parciales.

5. Modelo de Clases

5.1 Descripción

El modelo usa seis clases persistentes. **Comanda** representa el encabezado del servicio y **DetalleComanda** sus renglones. Los atributos **ocupada**, **subtotal** y **total** son propiedades calculadas, no columnas. Una comanda recién abierta puede no tener detalles; solo se exige al menos uno para cerrarla.

5.2 Diagrama de clases

La fuente única y editable está en **diagramas/modelo_clases.puml**. Debe renderizarse desde ese archivo para evitar mantener copias divergentes.

5.3 Responsabilidades

Clase	Responsabilidad principal
Mesero	Identificar a quien atiende la comanda.
Mesa	Representar el lugar de servicio y exponer su disponibilidad.
Categoria	Agrupar productos para facilitar su selección.
Producto	Mantener el artículo vendible y su precio actual.
Comanda	Controlar el ciclo del servicio y calcular su total.
DetalleComanda	Registrar producto, cantidad, precio histórico y subtotal.

6. Modelo de Datos

6.1 Modelo relacional

Tabla	Llave primaria	Llaves foráneas	Relación
mesero	id	—	Un mesero atiende muchas comandas.
mesa	id	—	Una mesa tiene muchas comandas históricas.
categoria	id	—	Una categoría clasifica muchos productos.

Tabla	Llave primaria	Llaves foráneas	Relación
producto	id	categoria_id	Un producto pertenece a una categoría.
comanda	id	mesa_id, mesero_id	Una comanda pertenece a una mesa y un mesero.
detalle_comanda	id	comanda_id, producto_id	Una comanda contiene varios detalles.

6.2 Restricciones e índices recomendados

- Índice por `comanda(estado)` para consultar comandas abiertas.
- Índice por `comanda(mesa_id, estado)` para determinar rápidamente la ocupación.
- Restricción `UniqueConstraint(fields=['mesa'], condition=Q(estado='ABIERTA'))` para garantizar una sola comanda abierta por mesa. La vista también debe validarlo para ofrecer un mensaje comprensible.
- Restricción única sobre `detalle_comanda(comanda_id, producto_id)` si se decide acumular productos iguales.
- Restricciones `CHECK` para `precio >= 0`, `precio_unitario >= 0`, `cantidad > 0` y `capacidad > 0`.
- La apertura y el cierre deben ejecutarse dentro de `transaction.atomic()`. La validación de la aplicación no sustituye la restricción ante solicitudes concurrentes.

6.3 Decisiones de diseño

- No se almacena `mesa.ocupada`; se deriva de las comandas abiertas.
- En el MVP no se almacenan subtotales ni total, pues se calculan desde los detalles. Un eventual `total_cierre` histórico deberá guardarse en la misma transacción de cierre.
- `precio_unitario` sí se almacena en el detalle para conservar el precio histórico.
- Desactivar catálogos es preferible a eliminarlos, ya que pueden estar relacionados con tickets anteriores.
- Las entidades referenciadas usan `on_delete=PROTECT`. `DetalleComanda.comanda` usa `CASCADE`, pero la interfaz y Django Admin deben impedir eliminar comandas cerradas para conservar el historial.

6.4 Diagrama entidad-relación

La fuente única y editable está en `diagramas/modelo_datos.puml`. Debe renderizarse desde ese archivo para evitar mantener copias divergentes.

7. Diccionario de Datos

Convenciones: **PK** = llave primaria, **FK** = llave foránea, **UQ** = valor único, **NN** = no nulo. Django crea el campo `id` automáticamente si no se declara otro identificador.

7.1 Tabla `mesero`

Campo	Tipo Django / SQL	Restricciones	Descripción
id	BigAutoField / BIGINT	PK	Identificador del mesero.

Campo	Tipo Django / SQL	Restricciones	Descripción
nombre	CharField(100) / VARCHAR(100)	NN	Nombre mostrado en la selección.
activo	BooleanField / BOOLEAN	NN, default True	Indica si puede seleccionarse.

7.2 Tabla mesa

Campo	Tipo Django / SQL	Restricciones	Descripción
id	BigAutoField / BIGINT	PK	Identificador de la mesa.
numero	PositiveIntegerField / INTEGER	NN, UQ, mayor que 0	Número visible de la mesa.
capacidad	PositiveIntegerField / INTEGER	NN, mayor que 0	Número orientativo de comensales.
activa	BooleanField / BOOLEAN	NN, default True	Indica si la mesa está en servicio.
ocupada	Propiedad calculada	No persistente	Verdadero si existe una comanda abierta.

7.3 Tabla categoria

Campo	Tipo Django / SQL	Restricciones	Descripción
id	BigAutoField / BIGINT	PK	Identificador de la categoría.
nombre	CharField(80) / VARCHAR(80)	NN, UQ	Ejemplo: entradas, bebidas o postres.
activa	BooleanField / BOOLEAN	NN, default True	Indica si se muestra al capturar.

7.4 Tabla producto

Campo	Tipo Django / SQL	Restricciones	Descripción
id	BigAutoField / BIGINT	PK	Identificador del producto.
categoria_id	ForeignKey / BIGINT	FK, NN, PROTECT	Categoría a la que pertenece.
nombre	CharField(120) / VARCHAR(120)	NN	Nombre del platillo o bebida.
precio	DecimalField(10,2) / DECIMAL(10,2)	NN, mayor o igual a 0	Precio actual de venta.
activo	BooleanField / BOOLEAN	NN, default True	Indica si se puede agregar a comandas.

Se recomienda la unicidad conjunta de `categoria_id` y `nombre`.

7.5 Tabla comanda

Campo	Tipo Django / SQL	Restricciones	Descripción
id	BigAutoField / BIGINT	PK	Folio interno de la comanda/ticket.
mesa_id	ForeignKey / BIGINT	FK, NN, PROTECT	Mesa atendida.
mesero_id	ForeignKey / BIGINT	FK, NN, PROTECT	Mesero responsable al abrirla.
estado	CharField(10) / VARCHAR(10)	NN, valores ABIERTA o CERRADA	Estado del servicio.
fecha_apertura	DateTimeField / DATETIME	NN, auto al crear	Momento de inicio del servicio.
fecha_cierre	DateTimeField / DATETIME	Nulo permitido	Momento de cierre; nulo mientras está abierta.
total	Propiedad calculada	No persistente	Suma de los subtotales de los detalles; se calcula también al presentar un ticket cerrado.

7.6 Tabla detalle_comanda

Campo	Tipo Django / SQL	Restricciones	Descripción
id	BigAutoField / BIGINT	PK	Identificador del renglón.
comanda_id	ForeignKey / BIGINT	FK, NN, CASCADE	Comanda a la que pertenece.
producto_id	ForeignKey / BIGINT	FK, NN, PROTECT	Producto solicitado.
cantidad	PositiveIntegerField / INTEGER	NN, mayor que 0	Unidades solicitadas.
precio_unitario	DecimalField(10,2) / DECIMAL(10,2)	NN, mayor o igual a 0	Copia del precio al agregar el producto.
subtotal	Propiedad calculada	No persistente	cantidad × precio_unitario.

7.7 Datos iniciales sugeridos

Para evitar consumir tiempo del taller en captura manual, se recomienda preparar:

- Tres meseros activos.
- Seis mesas con distintas capacidades.
- Categorías: Entradas, Platos fuertes, Bebidas y Postres.
- Entre ocho y doce productos con precios sencillos.

Documentos complementarios

- `RUTAS_Y_PANTALLAS.md`: navegación, métodos HTTP y formularios.

Los archivos `.puml` de `diagramas/` son la fuente editable oficial de los diagramas.